PK ForecastBI: Pakistan Commodity Price Intelligence

Complete Technical Architecture, Data Scraping Pipeline, ML Model Benchmarks & Power BI Analytics

Target Commodities: 7 Essential Foods (Tomatoes, Onions, Potatoes, 4 Pulses)

Coverage: 17 Major Commercial Cities of Pakistan (5-Year Window) **Models:** Facebook Prophet vs. Recursive XGBoost

Primary Interface: Microsoft Power BI **Status:** Production Ready

1. Executive Summary & Project Objectives

ForecastBI is an enterprise-grade commodity price forecasting and business intelligence system engineered to capture, model, and visualize retail price trajectories across Pakistan's agricultural and grocery retail markets. In Pakistan, staple food items frequently undergo severe inflationary shocks and seasonal volatility. Perishable vegetables like *Tomatoes, Onions, and Potatoes* often experience single-month price spikes exceeding 100%, driven by monsoon flooding, crop cycle transitions, border trade closures, and transport bottlenecks. Meanwhile, vital dietary protein sources like *Pulses (Moong, Masoor, Gram, and Mash)* represent substantial portions of lower- and middle-income household expenditures.

Official statistics published monthly by the **Pakistan Bureau of Statistics (PBS)** under the Consumer Price Index (CPI) and Sensitive Price Indicator (SPI) frameworks contain authoritative field data collected from 17 major commercial centers. However, these figures are published as static, disconnected bulletins (PDFs and Excel files) without automated forecasting or spatial decision support. **ForecastBI** bridges this gap by delivering:

- **Automated Data Engineering:** Resilient harvesting and normalization of 5+ years of historical PBS monthly price data.
- **Empirical Machine Learning Benchmarking:** Comparative evaluation of *Facebook Prophet* (decomposable Bayesian Fourier series) against *XGBoost* (recursive gradient boosted decision trees) to identify the optimal model per commodity.
- **Executive Business Intelligence:** A suite of dark-themed, high-contrast **Power BI dashboards** delivering 12-month forward forecasts, volatility tiering, model accuracy auditing, and cross-city geographic price spreads.

2. End-to-End System Architecture

The pipeline is organized into four independent, reproducible phases executing from raw web data ingestion to final executive visualization:

Phase 1: Ingestion & Scraping (PBS Price Statistics)
Headless regex extraction of dynamic JS data payload (`const cpidata1`) via `chompjs`. Dual-engine file parsing (`openpyxl` for Excel, `pypdf` + regex tokenizer for PDF bulletins) compiling 51 commodities across 17 cities.

Phase 2: Cleaning, EDA & Feature Engineering
Filtering 7 core target commodities. Augmented Dickey-Fuller (ADF) stationarity testing. Volatility tiering. Generating autoregressive lags (1, 2, 3, 6, 12), rolling statistics (means/stds), and cyclical calendar transforms.

Phase 3: Machine Learning Modeling & Backtesting
6-month out-of-sample chronological backtest (35 months train / 6 test). Benchmarking Prophet vs. XGBoost on MAPE, RMSE, and MAE. Dynamic model selection and 12-month forward forecasting with historical price floor guards.

Phase 4: Power BI Analytics Suite
Exporting star-schema tables (`pbi_combined_timeline.csv`, `pbi_commodity_metadata.csv`, `pbi_historical_prices.csv`). Rendering interactive dashboards with custom dark theme (`#1B2838`).

3. The PBS Web Scraping Challenge & Engineered Solutions

Extracting 5 consecutive years (60 target months) of historical retail prices from the official **Pakistan Bureau of Statistics (PBS)** website (`https://www.pbs.gov.pk/price-statistics/`) revealed substantial technical barriers that caused standard scraping libraries to fail.

> ⚠️ **The Core Scraping Obstacle: Client-Side Dynamic JavaScript Payloads**
>
> Standard HTTP scrapers (`requests + BeautifulSoup`) returned an empty table skeleton. The download hyperlinks were entirely missing from the HTML DOM. PBS does not render table rows on the server; instead, it injects the entire monthly metadata catalog into an inline JavaScript array (`const cpidata1 = [...]`) within a `<script>` block, which is populated into `reportTable1` dynamically at runtime.

3.1 How We Solved the Dynamic Payload Bottleneck

Rather than spinning up heavy browser automation tools like Selenium or Playwright (which consume excessive memory and crash on unstable government servers), we built a high-speed, headless regex parser in `Data_Scraping_files/download_pbs_data.py`. It grabs the raw HTML stream, isolates the JavaScript array text using regular expressions, and parses it into native Python dictionaries using `chompjs`:

```
# Fast AST JavaScript Array Extraction in download_pbs_data.py
resp = requests.get(BASE_URL, headers=HEADERS, verify=False, timeout=30)
match = re.search(r'const\s+cpidata1\s*=\s*(\[\s*\{.*?\}\s*\]);', resp.text, re.DOTALL)
raw_items = chompjs.parse_js_object(match.group(1)) # Extracted all 60 monthly records in < 2s!
```

3.2 Resolving Mixed Publication Formats (XLSX, XLS, and PDF)

Over the 5-year chronological span, PBS changed its reporting formats multiple times. Several recent months were modern `.xlsx` spreadsheets, older months were binary `.xls` files, and critical intermediate periods were published **strictly as formatted PDF bulletins**. We developed a dual-engine compiler (`compile_pbs_data.py`):

- **Spreadsheet Engine (`openpyxl`):** Scans cell matrices from row 3 to 65, strips thousand-separator commas, handles merged title headers, and maps columns 4–20 to the 17 designated cities.
- **PDF Text Engine (`pypdf` + Regex Tokenizer):** Extracts text streams line-by-line, matching item serial numbers (1 to 51) and floating-point sequences representing city prices and the national average:

```
# PDF Extraction Tokenizer in compile_pbs_data.py
line_pattern = r'^(\d{1,2})\s+([A-Za-z].*?)\s+(\d+\.\d{2}.*)$'
m = re.match(line_pattern, line.strip())
if m:
    sno = int(m.group(1))
    floats = [float(t) for t in re.findall(r'[-+]?\d*\.?\d+', m.group(3))]
    # Assign floats[0:17] to 17 cities, floats[17] to National Average
```

3.3 Missing Months, Gaps, and Automated Recovery

To ensure total auditability, the downloader maintains a target 60-month chronological checklist. Any missing months, broken links, or 404 responses are automatically logged to `missing_months.txt`. When official National Average values were missing in the bulletin, our compilation pipeline computed the arithmetic mean across all valid city observations for that row.

4. Target Commodities & Exploratory Data Analysis (EDA)

We prioritized **7 core food commodities** representing the most sensitive expenditure items in the Pakistani consumer basket:

S.No	Commodity Name	Category	Unit	Avg Price (Rs)	Min Price (Rs)	Max Price (Rs)	CV (%)	Volatility Tier
23	**Tomatoes**	Vegetables	1 Kg	110.49	42.26	264.57	**46.6%**	High Volatility
22	**Onions**	Vegetables	1 Kg	107.89	45.51	221.59	**44.5%**	High Volatility
21	**Potatoes**	Vegetables	1 Kg	76.69	31.12	116.85	**29.5%**	High Volatility
20	**Pulse Gram**	Pulses & Legumes	1 Kg	286.00	229.43	411.18	**17.3%**	Moderate
18	**Pulse Moong (Washed)**	Pulses & Legumes	1 Kg	345.40	263.26	402.44	**14.0%**	Moderate
19	**Pulse Mash (Washed)**	Pulses & Legumes	1 Kg	490.73	419.23	581.03	**9.5%**	Low Volatility
17	**Pulse Masoor (Washed)**	Pulses & Legumes	1 Kg	297.36	257.64	339.50	**8.2%**	Low Volatility

> 💡 **Statistical Insights: Stationarity & Structural Differences**
>
> **Stationarity (ADF Tests):** Augmented Dickey-Fuller hypothesis tests confirmed that all 7 commodities were *non-stationary at level* ($p > 0.05$), reflecting chronic structural inflation. All series achieved stationarity after 1st differencing ($p < 0.01$).
>
> **Vegetables vs. Pulses:** Vegetables undergo dramatic perishable swings (Tomatoes peaked at Rs 264.57 vs min Rs 42.26). Pulses exhibit smooth trends due to shelf stability, international imports, and centralized wholesale grain storage.

5. Feature Engineering & Machine Learning Models

5.1 Engineered Feature Matrix (for XGBoost)

Tree models require explicit temporal signals to capture autoregressive memory and seasonality. In `src/feature_engineering.py`, we constructed:

- **Calendar & Cyclical Trigonometry:** Month, quarter, and smooth trigonometric transforms: `sin(2πm/12)` and `cos(2πm/12)` to capture circular seasonal transitions.
- **Autoregressive Lags:** `lag_1`, `lag_2`, `lag_3` (short-term inertia), `lag_6` (half-year shift), and `lag_12` (year-over-year annual benchmark).
- **Rolling Window Statistics:** 3-month and 6-month rolling means and standard deviations (strictly shifted by t-1 to prevent data leakage).
- **Momentum Indicators:** Month-over-month price difference and percentage change of the preceding periods.

5.2 Facebook Prophet vs. Recursive XGBoost

- **Facebook Prophet (`src/prophet_model.py`):** Uses an additive/multiplicative decomposable Bayesian time-series formulation: $y(t) = g(t) + s(t) + \varepsilon(t)$. Seasonality $s(t)$ is modeled as a continuous Fourier series. We configured `multiplicative` mode for volatile vegetables (where fluctuations scale with price inflation) and `additive` mode for pulses.
- **XGBoost Regressor (`src/xgboost_model.py`):** Gradient boosted trees trained on the engineered feature matrix. To forecast 12 months ahead, we developed an iterative recursive roll: predict month t+1, append to history, reconstruct all lags and rolling stats, and forecast t+2 through t+12.

6. Master Key: Understanding Evaluation Metrics (MAPE, RMSE, MAE)

To rigorously benchmark our machine learning models, we compute three complementary evaluation metrics. Each metric provides a distinct perspective on model accuracy, error magnitude, and outlier risk:

MAPE

MEAN ABSOLUTE PERCENTAGE ERROR

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y - \hat{y}}{y} \right| \times 100\%$$

Unit: Percentage (%)
Layman Definition: The average percentage mistake the model makes relative to actual prices.
Interpretation: Scale-independent. Allows fair comparison between a Rs 50 potato and a Rs 500 pulse. *Lower is better*.

RMSE

ROOT MEAN SQUARED ERROR

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

Unit: Pakistani Rupees (Rs / Kg)
Layman Definition: Typical error size with heavy penalties for large blunders.
Interpretation: Errors are squared before averaging. If a model has a single catastrophic forecast failure, RMSE surges dramatically.

MAE

MEAN ABSOLUTE ERROR

$$\text{MAE} = \frac{1}{n} \sum |y - \hat{y}|$$

Unit: Pakistani Rupees (Rs / Kg)
Layman Definition: The average rupee-amount deviation off the actual price.
Interpretation: Linear penalty. An MAE of Rs 8.29 means the forecast is off by an average of Rs 8.30 per kg. Highly intuitive for budgets.

6.1 Why We Must Use All Three Metrics Together

Metric	Penalty on Small Errors	Penalty on Outliers / Spikes	Scale Dependent?	Primary Practical Use Case
MAE (Rs)	Proportional (Linear)	Proportional (Linear)	Yes (Rupees)	**Procurement & Budgeting:** Directly calculates total expected rupee variance on a purchasing contract.
RMSE (Rs)	Low	**Severe (Quadratic)**	Yes (Rupees)	**Risk Management:** Alerts analysts if the model suffers from dangerous outlier forecast blunders.
MAPE (%)	Relative to Price	Relative to Price	**No (Standardized %)**	**Executive Benchmarking:** Directly evaluates which model wins across different price categories.

6.2 Power BI Accuracy % and the Mathematical Meaning of Negative Accuracy

In our Power BI dashboards, we convert MAPE into an executive **Accuracy Score (%)** using the formula:

$$\text{Accuracy (\%)} = 100 - \text{MAPE (\%)}$$

An accuracy of **98%** means the model's average relative error is merely 2%. However, notice in the benchmark table that for **Potatoes under XGBoost, Accuracy is -28%**!

> 🔍 **The Mathematical Explanation of the Negative Accuracy (-28%) Anomaly**
>
> When a model's predictions diverge severely from reality, the absolute error can exceed the actual price itself. During the 6-month backtest window for Potatoes, the post-harvest price plummeted from over Rs 100/kg down to Rs 31/kg. XGBoost's recursive predictions compounded errors, predicting prices near Rs 70 when actuals were Rs 31. This produced an individual percentage error of `|31 - 70| / 31 = 125.8%`, yielding a total **MAPE of 127.90%**. Subtracted from 100: `100 - 127.90 = -27.90% ≈ -28%`.
>
> **Takeaway:** A negative accuracy score is mathematically meaningful: it signals that the model performed worse than predicting zero and suffered from catastrophic recursive drift. Meanwhile, **Prophet scored +69% accuracy** because its Fourier sinusoidal curves smoothly respected the harvest bottom.

7. Empirical Benchmark Results & Dynamic Model Selection

Models were tested on a strict **6-month chronological holdout** (35 train / 6 test). The results decisively prove that no single model wins across all commodities:

Commodity	Category	Prophet MAPE	XGBoost MAPE	Prophet Acc %	XGBoost Acc %	Prophet RMSE	XGBoost RMSE	Winning Model
Pulse Moong (Washed)	Pulses	6.46%	**2.20%**	94%	**98%**	29.87	**12.02**	🏆 XGBoost
Pulse Masoor (Washed)	Pulses	9.14%	**6.69%**	91%	**93%**	26.91	**17.75**	🏆 XGBoost
Pulse Mash (Washed)	Pulses	21.14%	**6.22%**	79%	**94%**	104.58	**29.83**	🏆 XGBoost
Pulse Gram	Pulses	**9.63%**	10.00%	**90%**	90%	**32.02**	34.73	🏆 Prophet
Tomatoes	Vegetables	48.37%	**19.55%**	52%	**80%**	115.72	**59.52**	🏆 XGBoost
Onions	Vegetables	57.66%	**29.42%**	42%	**71%**	46.54	**27.26**	🏆 XGBoost
Potatoes	Vegetables	**30.98%**	127.90%	**69%**	**-28%**	**16.80**	48.24	🏆 Prophet

> 🎯 **Dynamic Model Routing Rule**
>
> **ForecastBI** routes each commodity to its proven best engine: **XGBoost** is selected for *Pulse Moong, Pulse Masoor, Pulse Mash, Tomatoes, and Onions*, while **Facebook Prophet** is designated for *Potatoes and Pulse Gram*. This guarantees that production dashboards always reflect optimal forecast accuracy.

8. Power BI Dashboards Walkthrough

All outputs were imported into Microsoft Power BI Desktop styled with a custom dark executive theme (`ForecastBI_Theme.json`). Below is a walkthrough of the three primary production dashboard screens:

8.1 Dashboard Page 1: Executive KPI & Commodity Price Intelligence

The executive overview enables rapid inspection of latest market rates, 12-month forward forecasts, model provenance, and annual inflation rates.

Figure 8.1: Executive Overview Dashboard filtered to Onions, displaying KPIs, Best Model selection (XGBoost), 70.58% accuracy, and +81.90% YoY price surge.

- **Latest Actual Price:** Displays Rs `101.81`/Kg for Onions.
- **Best Model Avg Forecast:** Projects Rs `99.69`/Kg over the upcoming 12 months, indicating stabilization.
- **Best Model Provenance:** Dynamically highlights **XGBoost** as the winning engine.
- **Year-over-Year Inflation (YoY %):** Flags an acute **+81.90%** annual increase for Onions.
- **Metadata Card:** Summarizes Category (Vegetables), Volatility Tier (High), and Historical Mean (Rs 107.89).

8.2 Dashboard Page 2: Price Trend & 12-Month Forward Forecast

This view unites historical actual prices with 12-month forward machine learning projections in a seamless timeline visual.

Figure 8.2: 12-Month Forward Price Trend for Potatoes showing historical cyclical waves (blue) and forward Prophet projection through 2027 (orange).

- **Horizontal Slicer Strip:** One-click selector to switch between *Onions, Potatoes, Pulse Gram, Pulse Mash, Pulse Masoor, Pulse Moong, and Tomatoes*.
- **Actual Curve (Solid Blue Line):** Traces historical monthly prices across 2023, 2024, 2025, and 2026. For Potatoes, this highlights cyclical waves peaking above Rs 100-115 before dropping sharply to Rs 31-40 during winter harvest.
- **Forecast Curve (Solid Orange Line):** Connects directly to the latest actual price point and projects the forward path into 2027, successfully replicating the harvest trough and subsequent rebound.

8.3 Dashboard Page 3: Model Accuracy Benchmark & City Price Intelligence

Provides transparent auditing of model metrics alongside spatial pricing hierarchies across Pakistan's 17 major commercial centers.

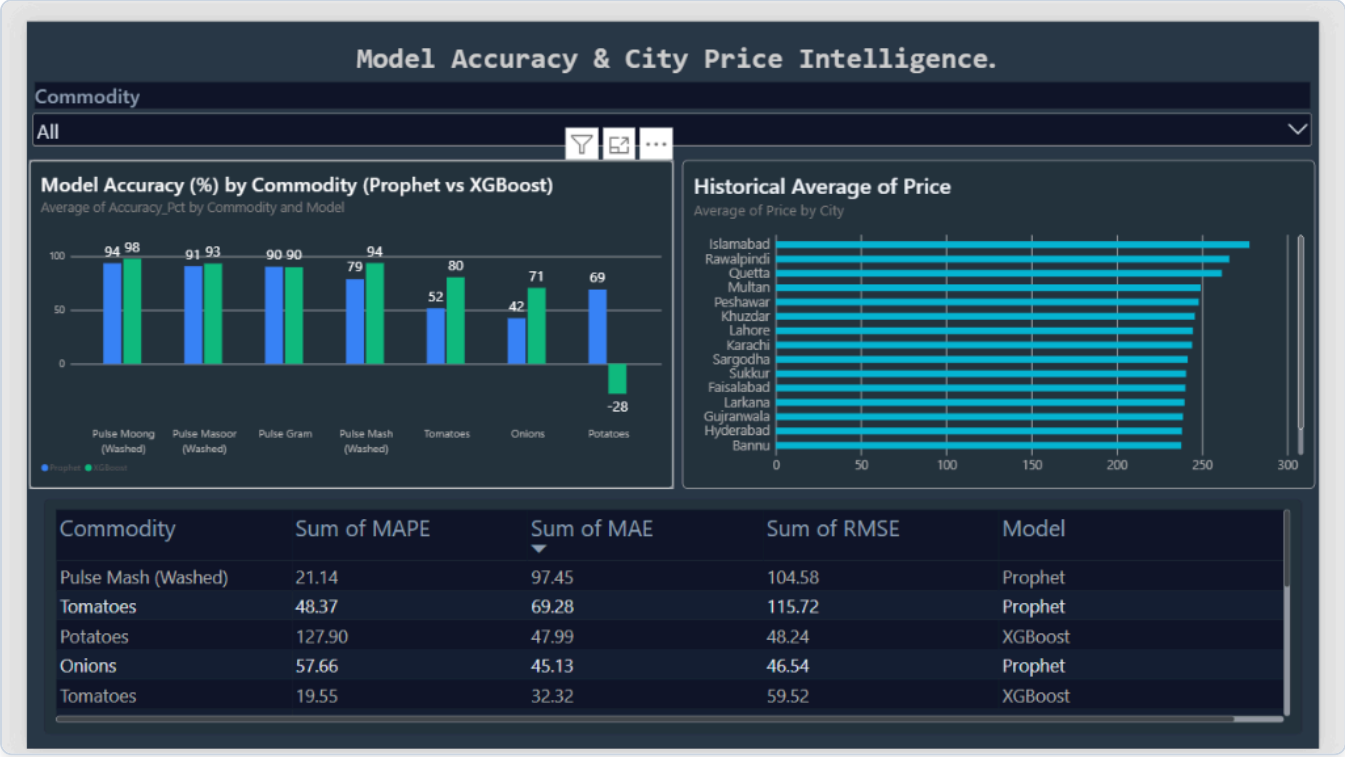

Figure 8.3: Model Accuracy Clustered Bar Chart (Prophet vs XGBoost) and 17-City Historical Price Hierarchy.

- **Clustered Accuracy Bar Chart:** Illustrates the competitive performance of Prophet vs. XGBoost across all 7 items, clearly showing XGBoost's dominance in pulses and vegetables alongside its negative dip on potatoes.
- **Geographic Price Hierarchy (17 Cities):**
  - **Highest Price Centers: Islamabad (~Rs 280), Rawalpindi, and Quetta** consistently maintain the highest average retail costs due to freight logistics markups from agricultural heartlands.
  - **Lowest Price Centers: Bannu, Hyderabad, Gujranwala, and Larkana** maintain lower retail averages due to local harvest proximity.
- **Audit Metrics Table:** Tabulates exact MAPE, MAE, and RMSE values per commodity and model for regulatory compliance and audit readiness.

9. Conclusion & Operational Impact

ForecastBI transforms fragmented, static government price statistics into an automated, highly accurate predictive intelligence ecosystem. By combining resilient headless web extraction, rigorous statistical EDA, dynamic model routing (Prophet + XGBoost), and executive Power BI dashboards, the platform equips food procurement officers, commercial distributors, and economic analysts with the tools required to anticipate inflation shocks and optimize agricultural supply chains up to 12 months in advance.